Лабораторная работа №7

Принципы SOLID и рефакторинг архитектуры Python-приложения

1. Цель работы

Применить принципы SOLID, разделить ответственность компонентов и сделать бизнес-логику расширяемой и тестируемой.

2. Постановка индивидуального задания

Вариант 8. Фитнес-центр. Система должна поддерживать регистрацию клиентов, оформление абонементов, запись на тренировку и отмену записи, предусматривать различные типы абонементов. Архитектура должна следовать принципам SOLID.

3. Краткие теоретические сведения

SRP разделяет ответственности, OCP позволяет расширять поведение без изменения устойчивого кода, LSP требует корректной подстановки типов, ISP рекомендует небольшие интерфейсы, DIP — зависимость от абстракций. В Python структурные интерфейсы удобно задавать через Protocol.

4. Исходный код программы

Файл interfaces.py

```
from typing import Protocol
class ClientRepository(Protocol):
    def add(self, client): ...
    def get(self, client_id): ...
    def get_all(self): ...
class Notifier(Protocol):
    def send(self, message: str): ...
class MembershipPricing(Protocol):
    def calculate(self, kind: str, months: int) -> float: ...
```

Файл main.py

```
from repositories import InMemoryClientRepository
from notifications import ConsoleNotifier
from pricing import StandardPricing, DiscountPricing, FamilyPricing
```

```
from services import FitnessService

def main():
    repository=InMemoryClientRepository(); notifier=ConsoleNotifier(); pricing=StandardPricing()
    service=FitnessService(repository,notifier,pricing)
    while True:
        print("\n=== Фитнес-центр (SOLID) ===")
        print("1. Регистрация клиента  2. Оформить абонемент  3. Запись на тренировку")
        print("4. Отменить запись  5. Список клиентов  6. Сменить тариф  0. Выход")
        choice=input("Выберите пункт: ").strip()
        try:
            if choice=="1": print(service.register_client(input("Имя: "),int(input("Возраст: "))))
            elif choice=="2": print(service.create_membership(int(input("ID клиента: ")),input("Тип
(разовый/месячный/годовой): ").strip()))
            elif choice=="3": print(service.book_training(int(input("ID клиента: ")),int(input("ID тренера: ")),input("Время:
").strip()))
            elif choice=="4": print(service.cancel_booking(int(input("ID записи: "))))
            elif choice=="5":
                for client in repository.get_all(): print(client)
            elif choice=="6":
                print("1. Стандартный 2. Скидка 3. Семейный")
                t=input("Тариф: ").strip(); service.pricing={"1":StandardPricing(),"2":DiscountPricing(),"3":FamilyPricing()}[t];
print("Тариф изменён.")
            elif choice=="0": break
            else: print("Неизвестный пункт меню.")
        except (ValueError,KeyError) as exc: print("Ошибка:",exc)
if __name__=="__main__": main()
```

Файл models.py

```
from dataclasses import dataclass
from enum import Enum
class MembershipStatus(Enum): ACTIVE="активен"; EXPIRED="истёк"; FROZEN="заморожен"; CANCELLED="отменён"
@dataclass
class Client:
    client_id:int; name:str; age:int
@dataclass
class Trainer:
    trainer_id:int; name:str; specialization:str
@dataclass
class Membership:
    membership_id:int; client_id:int; kind:str; price:float; status:MembershipStatus=MembershipStatus.ACTIVE
@dataclass
class TrainingBooking:
    booking_id:int; client_id:int; trainer_id:int; time:str
```

Файл notifications.py

```
class ConsoleNotifier:
    def send(self, message): print("[УВЕДОМЛЕНИЕ]", message)
class ListNotifier:
    def __init__(self): self.messages=[]
    def send(self, message): self.messages.append(message)
```

Файл pricing.py

```
class StandardPricing:
    def calculate(self, kind, months):
        prices={"разовый":500,"месячный":2500,"годовой":24000}
        if kind not in prices: raise ValueError("Неизвестный тип абонемента.")
        return prices[kind] if kind != "годовой" else prices[kind]
```

```
class DiscountPricing:
    def calculate(self, kind, months):
        return StandardPricing().calculate(kind, months) * 0.9
class FamilyPricing:
    def calculate(self, kind, months):
        return StandardPricing().calculate(kind, months) * 0.8
```

Файл repositories.py

```
class InMemoryClientRepository:
    def __init__(self): self._items = {}
    def add(self, client): self._items[client.client_id] = client
    def get(self, client_id): return self._items.get(client_id)
    def get_all(self): return list(self._items.values())
    def delete(self, client_id): return self._items.pop(client_id, None)
```

Файл services.py

```
from models import Client, Membership, TrainingBooking, MembershipStatus
class FitnessService:
    """Бизнес-логика зависит от абстракций репозитория, тарифа и уведомителя."""
    def __init__(self, repository, notifier, pricing):
        self.repository=repository; self.notifier=notifier; self.pricing=pricing
        self.memberships={}; self.bookings={}; self._next_client=1; self._next_membership=1; self._next_booking=1
    def register_client(self,name,age):
        if not name.strip() or age < 14: raise ValueError("Укажите имя; возраст от 14 лет.")
        client=Client(self._next_client,name.strip(),age); self._next_client+=1
        self.repository.add(client); self.notifier.send(f"Клиент {client.name} зарегистрирован")
        return client
    def create_membership(self,client_id,kind,months=1):
        if self.repository.get(client_id) is None: raise KeyError("Клиент не найден.")
        price=self.pricing.calculate(kind,months)
        membership=Membership(self._next_membership,client_id,kind,price); self._next_membership+=1
        self.memberships[membership.membership_id]=membership
        self.notifier.send(f"Оформлен абонемент №{membership.membership_id}")
        return membership
    def book_training(self,client_id,trainer_id,time):
        if self.repository.get(client_id) is None: raise KeyError("Клиент не найден.")
        booking=TrainingBooking(self._next_booking,client_id,trainer_id,time); self._next_booking+=1
        self.bookings[booking.booking_id]=booking; self.notifier.send(f"Запись на тренировку №{booking.booking_id}
создана")
        return booking
    def cancel_booking(self,booking_id):
        booking=self.bookings.pop(booking_id,None)
        if booking is None: raise KeyError("Запись не найдена.")
        self.notifier.send(f"Запись №{booking_id} отменена"); return booking
```

5. Объяснение работы программы

Модели описывают клиентов, тренеров, абонементы и записи. InMemoryClientRepository отвечает за хранение, FitnessService — за бизнес-операции. Уведомитель и стратегия цены передаются через конструктор, поэтому сервис можно тестировать с заменяемыми зависимостями.

6. Примеры работы программы в виде снимков экрана

```
=== Фитнес-центр (SOLID) ===
1. Регистрация клиента  2. Оформить абонемент  3. Запись на тренировку
4. Отменить запись  5. Список клиентов  6. Сменить тариф  0. Выход
Выберите пункт: 1
Имя: Петр
Возраст: 31
[УВЕДОМЛЕНИЕ] Клиент Петр зарегистрирован
Client(client_id=1, name='Петр', age=31)
```

Рисунок 7.1 – Регистрация клиента и вывод уведомления.

```
=== Фитнес-центр (SOLID) ===
1. Регистрация клиента  2. Оформить абонемент  3. Запись на тренировку
4. Отменить запись  5. Список клиентов  6. Сменить тариф  0. Выход
Выберите пункт: 2
ID клиента: 1
Тип (разовый/месячный/годовой): годовой
[УВЕДОМЛЕНИЕ] Оформлен абонемент №1
Membership(membership_id=1, client_id=1, kind='годовой', price=24000, status=<MembershipStatus.ACTIVE: 'активен'>)
```

Рисунок 7.2 – Оформление абонемента с выбранной стратегией цены.

```
=== Фитнес-центр (SOLID) ===
1. Регистрация клиента  2. Оформить абонемент  3. Запись на тренировку
4. Отменить запись  5. Список клиентов  6. Сменить тариф  0. Выход
Выберите пункт: 3
ID клиента: 1
ID тренера: 2
Время: 14:00
[УВЕДОМЛЕНИЕ] Запись на тренировку №1 создана
TrainingBooking(booking_id=1, client_id=1, trainer_id=2, time='14:00')

=== Фитнес-центр (SOLID) ===
1. Регистрация клиента  2. Оформить абонемент  3. Запись на тренировку
4. Отменить запись  5. Список клиентов  6. Сменить тариф  0. Выход
Выберите пункт: 4
ID записи: 1
[УВЕДОМЛЕНИЕ] Запись №1 отменена
TrainingBooking(booking_id=1, client_id=1, trainer_id=2, time='14:00')
```

Рисунок 7.3 – Запись и отмена тренировки, демонстрация разделения компонентов.

7. Вывод

В ходе выполнения лабораторной работы были реализованы требования индивидуального задания варианта 8. Модели описывают клиентов, тренеров, абонементы и записи. InMemoryClientRepository отвечает за хранение, FitnessService — за бизнес-операции. Уведомитель и стратегия цены передаются через конструктор, поэтому сервис можно тестировать с заменяемыми зависимостями. Программа разделена на логические компоненты, содержит проверку типичных ошибок и сопровождается автоматическими тестами.

Выполнил: Куршаков Егор Игоревич